CYBERMONKEY

A cyber-awareness card game wrapped in a tower defence run

TEAMMATE GUIDE

What the game is, how the rules work, how the code is organised, and how to run a demo.

Built with Phaser 4, TypeScript and Vite

A quick handover for the team

This guide describes the current MVP based on the project README, game rules and source files. The original challenge PowerPoints were not available to inspect while this guide was prepared, so the current challenge data in the project is treated as the source of truth.

1. The idea in one minute

CyberMonkey turns cyber-security planning into a short strategy game. Before attacks arrive, the player chooses security control cards under a spending limit. Those same controls are scored against four incident phases. Matching a chosen control on each phase's reveal earns a tower; towers then defend a fictional energy network from incoming "packet" enemies.

The learning result and the combat result are shown separately: official control points measure the chosen controls, short questions reward behaviour knowledge, and survival waves show how long the defence lasts. The tower defence is a game metaphor, not a simulation of how real security products work.

The play loop

Step	What the player does
1. Build a loadout	Spend the £3m budget on control cards. The selection must include Prevent, Detect and Respond.
2. Match phase cards	For each of four incident phases, inspect three sets of three revealed controls. Claim matching owned controls with positive phase points.
3. Earn and deploy towers	Each claimed match creates a tower. Its level equals that control's points for the current phase. Place all earned towers on free pads.
4. Rehearse and learn	Defend a gentle wave, then answer a phase question. Correct answers add learning points and small integrity bonuses.
5. Survive	After phase four, all towers carry into escalating survival waves. Launch waves until defeat, or bank the run at an intermission.

What makes it a cyber-awareness game?

- Players compare the costs, intended uses and limitations of security controls.
- The four incident stories give context to the scoring and questions.
- Explanations are shown after answers, including incorrect answers and timeouts.
- The final learning score combines control points and behaviour points; waves cleared are reported separately.

2. Rules and scoring

Purchasing and official control score

The player has a £3m budget. Cards have fixed costs and belong to Prevent, Detect or Respond. A balanced starter loadout is available for a guided demo. Once locked, the loadout cannot be changed during that run. The supplied scoring engine evaluates the chosen controls across all four phases. A control earns its official points even if it is absent from the matching reveal.

Matching and tower rewards

Each phase has nine revealed cards: the three highest-scoring controls in each category. Reveals are fixed for a phase, with ties following the order in the challenge data. This makes runs comparable. The player can claim each eligible match once per phase. A match with 1, 2 or 3 phase points creates a tower at that level; a zero-point match gives no tower. The same owned control can create another tower in a later phase.

Wave and question bonuses

Event	MVP effect
Correct phase question	+3 behaviour points; repairs up to 10 rehearsal integrity; adds +5 integrity to the start of survival.
Wrong answer or timeout	No bonus. The correct answer and explanation are revealed.
Rehearsal leak	Reduces integrity, but the rehearsal cannot end the run; integrity stays at least 1%.
Survival leak	Reduces integrity by a wave-dependent amount. At 0%, the run ends.
Survival intermission	Player may launch the next wave or finish and bank the current survival result.

Tower roles (fictional combat shorthand)

Control category	In-game role
Prevent	Focused pulse damage to the foremost enemy in range.
Detect	Slows and marks an enemy; marked targets take extra damage.
Respond	Deals damage to a small group around the target.

These roles are a theme choice for the game. They do not assert that a real Prevent, Detect or Respond control behaves like a weapon or tower.

3. How it is built

The project is a browser game. Vite serves the development build and bundles the production version; TypeScript contains the rules and UI logic; Phaser draws and animates the map, towers, shots and enemies. Readable HTML handles cards, questions, score panels and keyboard-accessible placement controls.

Part	Responsibility
Phaser 4	Canvas map, towers, enemies, range previews and shot effects.
TypeScript	Run state, card choices, phase progression, tower rewards, placement and UI events.
Vite	Local development server and production bundling.
Supplied challenge data	Controls, costs, phase scores, stories, questions and official scoring logic.
Browser storage	Stores the best survival record on this device/browser only.

Where to look in the source

File	What is there
src/core/run.ts	Main game state machine: shop, matching, deployment, questions, survival and scoring bonuses.
src/core/combat.ts	Enemy spawning, tower targeting, damage, wave difficulty and win/defeat transitions.
src/core/map.ts	Route geometry and numbered build-pad positions.
src/core/controls.mjs	Challenge-provided scoring engine.
src/data/challenge.json	Controls, phase stories and scores used by the MVP.
src/data/questions.json	Question bank, correct answers and explanations.
src/main.ts	HTML interface, user actions, timer and transitions between screens.
src/game/scenes/Game.ts	Phaser map and battle visuals, driven by the core simulation.
src/style.css	Responsive styling and focus states.
tests/rules.test.ts	Automated checks of game rules and a full run.

A useful design boundary

The core game logic is separate from Phaser drawing. That means rule tests can exercise the game without opening a browser, and the combat simulation advances in bounded time steps so a faster screen refresh does not make towers stronger.

4. Run and demo the MVP

Prerequisite: Node.js 22.18 or later (Node 24 is recommended for the TypeScript tests). From the CyberMonkey project directory:

Command	Purpose
npm install	Install the project's locked dependencies (first setup only).
npm run dev	Start the local Vite development server; open the localhost URL printed by the terminal.
npm test	Run the rules tests and simulated full playthrough.
npm run build	Type-check and make the production bundle in dist/.
npm run preview	Serve the production bundle locally.

Suggested short walkthrough

- Choose "Use a balanced starter loadout" and lock it.
- Use "Collect all matches", then auto-place the earned towers.
- Start the rehearsal wave; optionally pause or use 2× speed.
- Answer the phase question and continue through all four phases.
- In survival, launch a few waves, then finish at an intermission to show the summary (or continue until defeat).
- Restart and try a different loadout to demonstrate that the setup changes the result.

What is included in this MVP

- One local single-player map, four fixed matching phases, one supplied behaviour question per phase and three tower roles.
- Build-pad placement by clicking the map or using the equivalent HTML pad controls; auto-placement is also available.
- Pause and 2× speed for combat; a pausable question timer; local best survival score.

Known limits to explain honestly

- No multiplayer, cloud save, mid-run purchasing, tower sales, sound, or resume after refresh.
- One map and a fixed set of reveals/questions. The survival wave is launched manually at each intermission.
- Live combat is visual; screen-reader support is stronger for the HTML controls/cards than for the moving battle itself.
- Small screens are supported, but desktop is the intended demo surface.

5. Team handover and presentation

Before presenting

- Run npm test and npm run build on the team machine; confirm the browser demo starts from the current checkout.
- Have one person drive the walkthrough and another explain how loadout choice affects control scoring and towers.
- Agree on the scenario theme and explain the game's limits as a learning metaphor.
- Add team member names and record what the team reviewed or changed. OUR_GAME.md still labels the team name as something to confirm and team members as to fill in.

Suggested 30-second explanation

"CyberMonkey is a cyber-awareness tower defence game. You spend a £3m budget on security controls, then compare your choices against four incident phases. Matching a selected control earns a tower whose strength is tied to the phase score. Between phases, a short question reinforces the learning. Once all towers are deployed, escalating waves show how long your defence lasts. We keep the official control score separate from the fictional combat, so the game makes the controls easier to explore without claiming to model real-world security operations."

Files and source notes

This guide was prepared from the current CyberMonkey README, OUR_GAME.md, package.json and the game rules/combat modules. The repo README identifies the challenge data and scoring module as supplied challenge-pack content, and the base project as the Phaser Studio Vite TypeScript template. The original PowerPoint files were not inspected for this PDF; check the hand-in brief and challenge slides before final submission, especially for any requirements that are not represented in the current code.

Project location

/Users/jason/Desktop/StrathHack/team-18/CyberMonkey

Prepared as a teammate handover for the CyberMonkey MVP. Keep the game rules, supplied challenge material and the competition brief aligned as the project evolves.